

Memoria de la práctica 1

Sección A

Búsqueda en profundidad

Hemos implementado el código de BP de la siguiente manera:

```
structure.push((search_problem.getStartState(), []))
visited = []

while not structure.isEmpty():
    current_state, actions = structure.pop()

    if search_problem.isGoalState(current_state):
        return actions

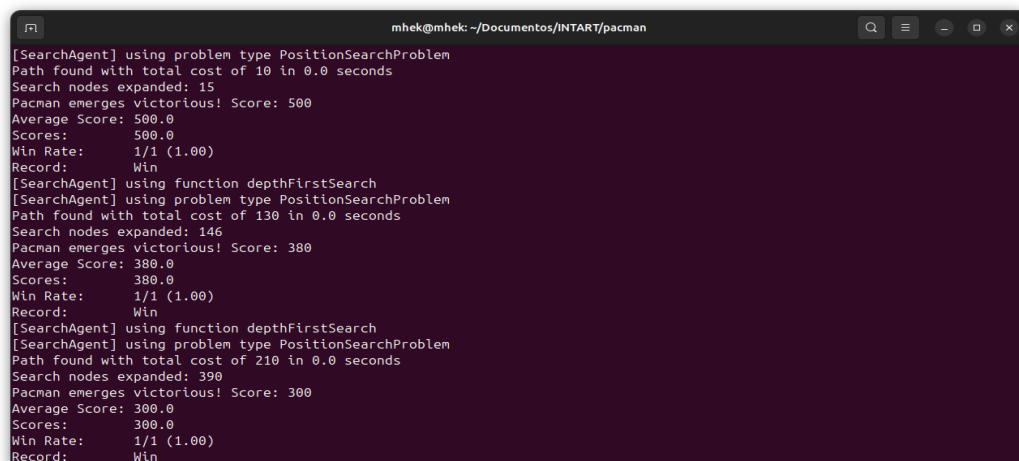
    if current_state not in visited:
        visited.append(current_state)

        for successor, action in search_problem.getSuccessors(current_state):
            if successor not in visited:
                new_actions = actions + [action]
                structure.push((successor, new_actions))

return None
```

La lista de abiertos (*structure*) la hemos inicializado con una tupla compuesta por dos elementos: el nodo y una lista de acciones que equivalen al camino previo realizado hasta llegar a dicho nodo, esta secuencia de acciones será los que devolvamos cuando lleguemos al estado objetivo.

Hacemos una primera comprobación del código para la búsqueda en profundidad con los laberintos:



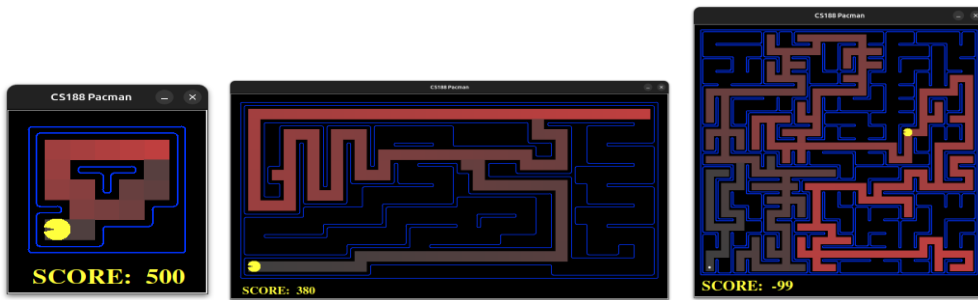
```
mhek@mhek: ~/Documentos/INTART/pacman

[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 10 in 0.0 seconds
Search nodes expanded: 15
Pacman emerges victorious! Score: 500
Average Score: 500.0
Scores: 500.0
Win Rate: 1/1 (1.00)
Record: Win

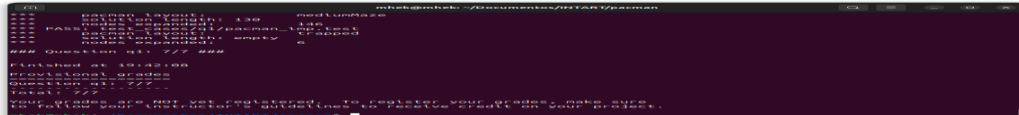
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 130 in 0.0 seconds
Search nodes expanded: 146
Pacman emerges victorious! Score: 380
Average Score: 380.0
Scores: 380.0
Win Rate: 1/1 (1.00)
Record: Win

[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 210 in 0.0 seconds
Search nodes expanded: 390
Pacman emerges victorious! Score: 300
Average Score: 300.0
Scores: 300.0
Win Rate: 1/1 (1.00)
Record: Win
```

Mapas para *tinyMaze*, *mediumMaze* y *bigMaze*:



Comprobación con autograder:



Pregunta A.1.1: ¿Qué significa que sea completo?

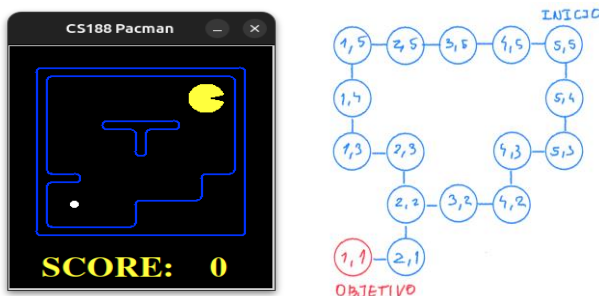
Un algoritmo completo es aquel que puede una solución en todos los casos. En el caso de BP no podemos hablar de un algoritmo completo 'real' porque si lo aplicamos en árboles profundidad infinita, el algoritmo se puede quedar atascado en un bucle infinito y no encontrar nunca la solución. Sin embargo, en la versión aplicada a grafos, que guarda los nodos visitados para no volver a expandirlos, sí es completo.

Pregunta A.1.2: ¿Si el algoritmo de BP permitiera expandir nodos ya visitados Pacman siempre puede llegar a la solución? ¿Por qué?

De acuerdo con nuestra respuesta a la pregunta anterior, si se permitieran expandir los nodos ya visitados obtendríamos la versión no completa del algoritmo y podríamos caer en bucles infinitos sin dar nunca con la solución.

Pregunta A.1.3: ¿El orden de exploración es el que esperabais? ¿Pacman realmente va a todas las casillas exploradas en su camino hacia la meta?

Utilizando el laberinto pequeño para poder comparar una ejecución manual del algoritmo BP con la implementada, tenemos el siguiente grafo según el mapa:



Sabiendo que la lista de abiertos para BP debe ser LIFO, si la inserción de los nodos en la lista puede ser en función de la coordenada X o la Y del punto que representan. El orden de exploración que nosotros esperábamos estaba pensado con una lista de nodos abiertos ordenada en función de la coordenada X, de tal manera que comenzaría

explorando el nodo (5,4) partiendo de la lista de abiertos [(4, 5), (5,4)] y la primera exploración hubiera sido por el lado derecho del laberinto, sin embargo, por cómo devuelve los sucesores de cada nodo la función `getSuccessors()`, la exploración comienza por el lado izquierdo del laberinto.

No, se exploran más nodos que los que tiene el camino que sigue pacman para llegar a la meta.

Pregunta A.1.4: ¿La solución que encuentra Pacman al usar BP es la solución de menor coste? ¿Por qué?

BP no es un algoritmo óptimo, por tanto, la solución encontrada con este algoritmo no tiene por qué ser la de menor coste. Si hay más de una solución, BP puede dar un camino subóptimo.

Búsqueda en anchura

Para adaptar el código desarrollado en el apartado anterior para implementar BA, simplemente hemos hecho que la función de BG reciba tanto el problema de búsqueda como la vista de nodos visitados, siendo esto último lo que varía en nuestras implementaciones de BP (una pila) y BA (una cola). De esta manera, el código de BA se reduce a crear la instancia de una cola y llamar a BG.

```
def depthFirstSearch(search_problem):
    # Import pdb; pdb.set_trace()
    return genericSearch(search_problem, util.Stack())

def genericSearch(search_problem, structure):
    ...

def breadthFirstSearch(search_problem):
    """ YOUR CODE HERE """
    return genericSearch(search_problem, util.Queue())
```

```
def depthFirstSearch(search_problem):
    # Import pdb; pdb.set_trace()
    return genericSearch(search_problem, util.Stack())

def genericSearch(search_problem, structure):
    ...

def breadthFirstSearch(search_problem):
    """ YOUR CODE HERE """
    return genericSearch(search_problem, util.Queue())
```

```
def genericSearch(search_problem, structure):
    """
    structure.push((search_problem.getStartState(), [], 0))
    visited = []

    while not structure.isEmpty():
        current_state, actions = structure.pop()

        if search_problem.isGoalState(current_state):
            return actions

        if current_state not in visited:
            visited.append(current_state)

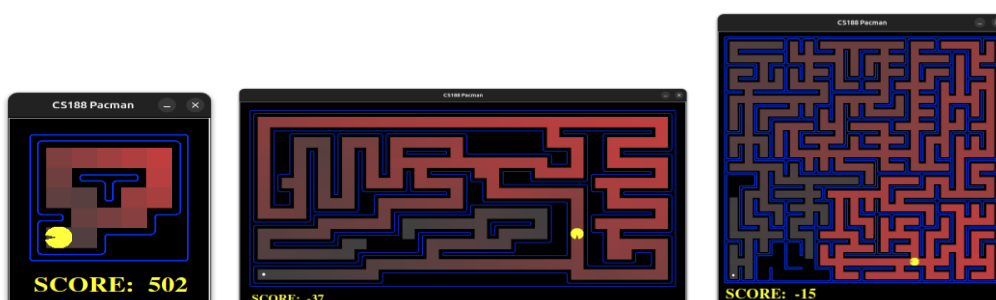
            for successor, action in search_problem.getSuccessors(current_state):
                if successor not in visited:
                    new_actions = actions + [action]
                    structure.push((successor, new_actions))

    return None
```

Comprobación del código implementado para la búsqueda en anchura con una búsqueda genérica basada en la primera implementación de BP (con los tres laberintos):

```
mhek@mhck: ~/Documentos/INTART/pacman
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 15
Pacman emerges victorious! Score: 502
Average Score: 502.0
Scores: 502.0
Win Rate: 1/1 (1.00)
Record: Win
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 68 in 0.0 seconds
Search nodes expanded: 269
Pacman emerges victorious! Score: 442
Average Score: 442.0
Scores: 442.0
Win Rate: 1/1 (1.00)
Record: Win
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 210 in 0.1 seconds
Search nodes expanded: 620
Pacman emerges victorious! Score: 300
Average Score: 300.0
Scores: 300.0
Win Rate: 1/1 (1.00)
Record: Win
mhek@mhck: ~/Documentos/INTART/pacman$
```

Mapas para *tinyMaze*, *mediumMaze* y *bigMaze*:



Comprobación con autograder:

[illegible]

Pregunta A.2.1: ¿BA encuentra la solución de menor coste? ¿Por qué?

Sí, porque explora todos los nodos que están en el mismo nivel de profundidad y busca el nodo objetivo a la profundidad menor.

Pregunta A.2.2: ¿En el peor caso de BA tendrá que expandir más nodos que en el peor caso de BP? ¿Por qué?

Si la solución se encuentra en la profundidad máxima del grafo, como BA tiene que explorar todos los nodos para un nivel de profundidad p , BA va a tener que expandir más nodos que el algoritmo de BP hasta encontrar la solución.

Pregunta A.2.3: ¿Por qué pacman cuando usa BA no entra en bucle, aunque haya bucles en el laberinto? ¿Cómo se llama a esta propiedad?

Porque BA cuenta una lista de nodos ya visitados para no volver a explorarlos, que permite la detección de ciclos en grafos; que es la propiedad de la búsqueda en anchura.

Búsqueda con coste uniforme

Para la búsqueda con coste uniforme hemos añadido un elemento más a la tupla de la lista de abiertos de BG, el coste, que en el estado inicial será cero.

```
def genericSearch(search_problem, structure):
    """ ...
    structure.push((search_problem.getStartState(), [], 0))
    visited = []

    while not structure.isEmpty():
        current_state, actions, cost = structure.pop()

        if search_problem.isGoalState(current_state):
            return actions

        if current_state not in visited:
            visited.append(current_state)

            for successor, action, s_cost in search_problem.getSuccessors(current_state):
                if successor not in visited:
                    new_actions = actions + [action]
                    new_cost = cost + s_cost
                    structure.push((successor, new_actions, new_cost))

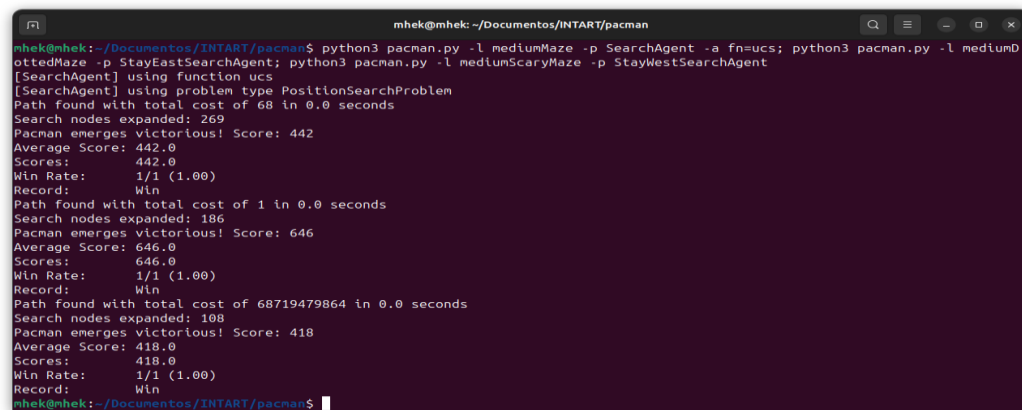
    return None
```

El código de la función de BCU se implementa una cola con función de prioridad. Esta función es una lambda que asigna la prioridad en base al coste del camino que hay hasta llegar al nodo que se va a añadir a la lista de abiertos.

```
def uniformCostSearch(search_problem):
    """Search the node of least total cost first."""
    """ YOUR CODE HERE """

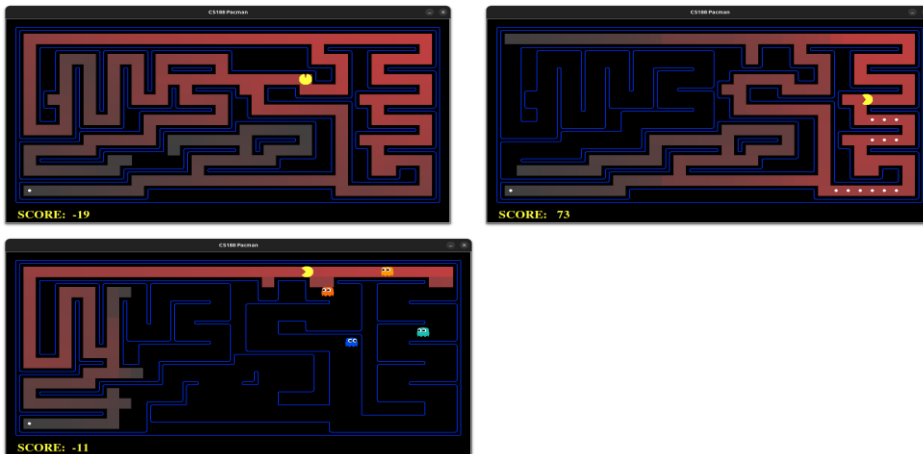
    #la prioridad viene dada por el tercer item de la tetrada, que es el coste del camino
    structure = util.PriorityQueueWithFunction(lambda item : item[2])
    return genericSearch(search_problem, structure)
```

Primera comprobación del código para la búsqueda con coste uniforme con los laberintos:

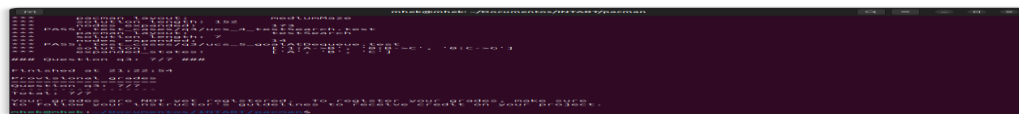


```
mhek@mhek: ~/Documentos/INTART/pacman
mhek@mhek: ~/Documentos/INTART/pacman$ python3 pacman.py -l mediumMaze -p SearchAgent -a fn=ucs; python3 pacman.py -l mediumDottedMaze -p StayEastSearchAgent; python3 pacman.py -l mediumScaryMaze -p StayWestSearchAgent
[SearchAgent] using function ucs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 68 in 0.0 seconds
Search nodes expanded: 269
Pacman emerges victorious! Score: 442
Average Score: 442.0
Scores: 442.0
Win Rate: 1/1 (1.00)
Record: Win
Path found with total cost of 1 in 0.0 seconds
Search nodes expanded: 186
Pacman emerges victorious! Score: 646
Average Score: 646.0
Scores: 646.0
Win Rate: 1/1 (1.00)
Record: Win
Path found with total cost of 68719479864 in 0.0 seconds
Search nodes expanded: 108
Pacman emerges victorious! Score: 418
Average Score: 418.0
Scores: 418.0
Win Rate: 1/1 (1.00)
Record: Win
mhek@mhek: ~/Documentos/INTART/pacman$
```

Mapas para *tinyMaze*, *mediumMaze* y *bigMaze*:



Comprobación con autograder:



Pregunta A.3.1: Si asignamos el mismo coste a todos los movimientos, ¿BCU expande los mismos nodos que BA?

Sí, porque el camino de menor coste y el de menor profundidad coincidirían, por lo que BCU se comportaría como BA.

Pregunta A.3.2: ¿Qué coste hay que asignar a cada nuevo nodo en BCU para que se comporte como BP?

Para que BCU se comporte como BP, habría que designar los costes de los nodos en función de cómo deberían explorarse si la lista de abiertos fuera una pila (LIFO) en lugar de una cola (FIFO).

Búsqueda con A*

Para búsqueda con heurística hemos utilizado la misma estructura de BG utilizando una cola de prioridad simple. La prioridad de los nodos está definida por el valor de la función heurística más su coste, que se calcula cuando se expande el nodo que estamos visitando.

```
def aStarSearch(search_problem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and heuristic first."""
    """ YOUR CODE HERE """
    #structure = util.PriorityQueueWithFunction(lambda item : item[2] + heuristic(item[1], search_problem))
    # Push inicial: path con el estado inicial, acciones vacías, costo 0
    structure = util.PriorityQueue()
    start_state = search_problem.getStartState()
    structure.push([start_state], [], 0, heuristic(start_state, search_problem))

    visited = []
    #import pdb; pdb.set_trace()
    while not structure.isEmpty():
        path, actions, cost = structure.pop()
        current_state = path[-1]

        if search_problem.isGoalState(current_state):
            return actions

        if current_state not in visited:
            visited.append(current_state)

        for successor, action, c in search_problem.getSuccessors(current_state):
            if successor not in visited:
                new_path = path + [successor]
                new_actions = actions + [action]
                new_cost = cost + c # costo acumulado real
                priority = new_cost + heuristic(successor, search_problem) # f = g + h
                structure.push((new_path, new_actions, new_cost), priority)

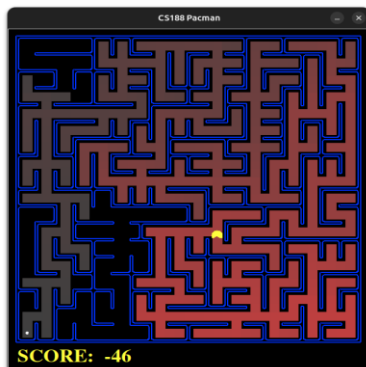
    return None #genericSearch(search_problem, structure)
```

Comprobación con el laberinto *bigMaze*:

```
Finished at 21:22:54
Provisional grades
=====
Question q4: 3/3
-----
Total: 3/3

Your grades are NOT yet registered. To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.
https://www.cs.berkeley.edu/~dave/CS188/

[Note: The above message is a placeholder for the actual content of the terminal window.]
```



Comprobación con autograder:

```
Provisional grades
=====
Question q4: 3/3
-----
Total: 3/3

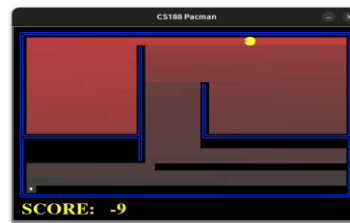
Your grades are NOT yet registered. To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.
```

Pregunta A.4.1: ¿Qué sucede en openMaze para las diversas estrategias de búsqueda?

Primero vamos a realizar las ejecuciones de todos los algoritmos de búsqueda para poder analizar su comportamiento.

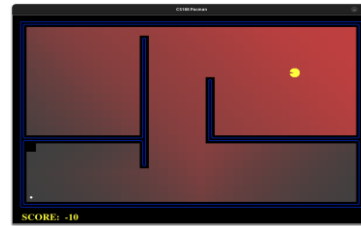
BP:

```
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 298 in 0.1 seconds
Search nodes expanded: 576
Pacman emerges victorious! Score: 212
Average Score: 212.0
Scores:      212.0
Win Rate:    1/1 (1.00)
Record:      Win
```



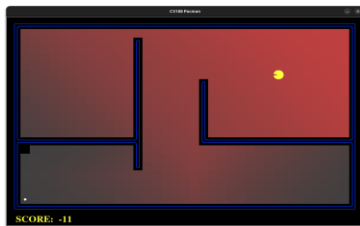
BA:

```
[SearchAgent] using function bfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 54 in 0.1 seconds
Search nodes expanded: 682
Pacman emerges victorious! Score: 456
Average Score: 456.0
Scores:      456.0
Win Rate:    1/1 (1.00)
Record:      Win
```



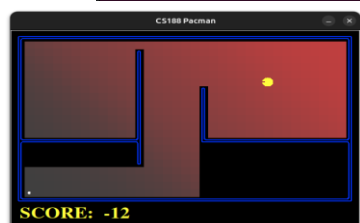
```
[SearchAgent] using function ucs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 54 in 0.1 seconds
Search nodes expanded: 682
Pacman emerges victorious! Score: 456
Average Score: 456.0
Scores:      456.0
Win Rate:    1/1 (1.00)
Record:      Win
```

BCU:



A*:

```
[SearchAgent] using function astar and heuristic manhattanHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 54 in 0.0 seconds
Search nodes expanded: 535
Pacman emerges victorious! Score: 456
Average Score: 456.0
Scores:      456.0
Win Rate:    1/1 (1.00)
Record:      Win
```



De todos los algoritmos de búsqueda, el único que no es óptimo, BP, ha encontrado una solución subóptima; y dada la naturaleza del algoritmo, es de los que menos nodos ha expandido junto con A*. Si nos fijamos en los algoritmos que sí son óptimos, tanto BCU como BA son los que más nodos expanden (682) y exploran casi todo el mapa para encontrar el camino más corto. A* ha encontrado el camino óptimo con el

menor número de nodos, 576, como BP; aunque a diferencia de este último, la elección de los nodos a explorar ha sido más eficiente, según muestran los mapas con las exploraciones de los algoritmos.

Pregunta A.4.2: ¿Qué cambia en el comportamiento de A* al usar la distancia de manhattan o la distancia euclídea?

Si aplicamos A* con distancia euclídea para el laberinto *openMaze* el número de nodos explorados se reduce ligeramente, pasando de 576 a 550 y los nodos explorados tienen patrón algo distinto:

```
[SearchAgent] using function astar and heuristic euclideanHeuristic
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 54 in 0.1 seconds
Search nodes expanded: 550
Pacman emerges victorious! Score: 456
Average Score: 456.0
Scores:      456.0
Win Rate:    1/1 (1.00)
Record:      Win
```



Estas diferencias en la última parte del laberinto se deben a que hay nodos cuya distancia euclídea es menor a la distancia de manhattan desde el nodo en el que se exploran.

Corners' Problem

Lo primero que hicimos para implementar el problema de las esquinas fue definir nuestro estado inicial y cuándo es un estado el objetivo.

```
def getStartState(self):
    """
    *** YOUR CODE HERE ***
    """
    return (self.startingPosition, set())
```

Hemos pensado que lo más conveniente para este problema es que el estado sea una tupla con las coordenadas y un set con las esquinas visitadas. Por ello, la función *isGoalState* está implementada con comparación de sets para comprobar si un estado es la meta o no:

```
def isGoalState(self, state):
    """
    *** YOUR CODE HERE ***
    """
    return state[1] == set(self.corners)
```

Esta implementación permite resolver problemas un número de esquinas cualesquiera. Una vez definidos los estados iniciales y finales, procedimos a implementar la búsqueda de sucesores, para lo cual, tomamos como referencia la búsqueda de sucesores con del problema de búsqueda posicional:

```

def getSuccessors(self, received_state):
    """ ...

    successors = []
    for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
        """ YOUR CODE HERE """
        x, y = received_state[0]
        dx, dy = Actions.directionToVector(action)
        nextx, nexty = int(x + dx), int(y + dy)

        if not self.walls[nextx][nexty]:
            visited_corners = set(received_state[1])

            if (nextx, nexty) in set(self.corners):
                visited_corners.add((nextx, nexty))

            nextState = ((nextx, nexty), visited_corners)
            successors.append((nextState, action, 1))

    self._expanded += 1 # DO NOT CHANGE
    return successors

```

Comprobación con los comandos `python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem` y `python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem`

```

[SearchAgent] using function bfs
[SearchAgent] using problem type CornersProblem
Path found with total cost of 28 in 0.0 seconds
Search nodes expanded: 252
Pacman emerges victorious! Score: 512
Average Score: 512.0
Scores:      512.0
Win Rate:    1/1 (1.00)
Record:      Win
[SearchAgent] using function bfs
[SearchAgent] using problem type CornersProblem
Path found with total cost of 106 in 0.3 seconds
Search nodes expanded: 1966
Pacman emerges victorious! Score: 434
Average Score: 434.0
Scores:      434.0
Win Rate:    1/1 (1.00)
Record:      Win

```

Comprobación con autograder:

```

Provisional grades
=====
Question q2: 7/7
Question q5: 3/3
-----
Total: 10/10

```

Heurística para Corners' Problem

Código de la heurística:

```

# heuristic value h(n)
# import pdb; pdb.set_trace()
h = 0
coords = cur_state[0]

pending_corners = list(set(corners).difference(cur_state[1]))
if (not pending_corners):
    return 0

# we analyze all the possible paths for the unvisited corners
min_dist = float('inf')
# we can make ourselves the permutations, but itertools lib has a function for that already
for path in itertools.permutations(pending_corners):
    dist = util.manhattanDistance(coords, path[0])
    for i in range(len(path) - 1):
        dist += util.manhattanDistance(path[i], path[i+1])
    if dist < min_dist:
        min_dist = dist
h += min_dist
return h

```

Comprobaciones con laberintos para el problema de las esquinas:

```

mhek@mhek:~/Documentos/INTART/pacman$ python3 pacman.py -l tinyCorners -p AStarCornersAgent -z 0.5
Path found with total cost of 28 in 0.0 seconds
Search nodes expanded: 159
Pacman emerges victorious! Score: 512
Average Score: 512.0
Scores: 512.0
Win Rate: 1/1 (1.00)
Record: Win

mhek@mhek:~/Documentos/INTART/pacman$ python3 pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
Path found with total cost of 106 in 0.1 seconds
Search nodes expanded: 741
Pacman emerges victorious! Score: 434
Average Score: 434.0
Scores: 434.0
Win Rate: 1/1 (1.00)
Record: Win

mhek@mhek:~/Documentos/INTART/pacman$ python3 pacman.py -l bigCorners -p AStarCornersAgent -z 0.5
Path found with total cost of 162 in 0.2 seconds
Search nodes expanded: 1725
Pacman emerges victorious! Score: 378
Average Score: 378.0
Scores: 378.0
Win Rate: 1/1 (1.00)
Record: Win

```

Comprobación con autograder:

```

Provisional grades
=====
Question q4: 3/3
Question q6: 6/6
-----
Total: 9/9

```

Pregunta A.6.1: Describe el proceso que habéis seguido para diseñar la heurística y explica la lógica de la misma.

Nuestra heurística calcula $h(n)$ la suma de las distancias de Manhattan entre las esquinas que queden por visitar a través del camino más corto desde el estado actual. El camino más corto lo obtenemos calculando la distancia de manhattan de todas combinaciones posibles que se puedan hacer con las esquinas restantes.

Sección B

Pregunta B.1.1: ¿Cómo se comporta cada método de búsqueda con cada uno de los problemas definidos?

[illegible][illegible]

Tras ver las ejecuciones de Othello.py no parece que se vea ninguna diferencia en cada una de las heurísticas utilizadas (min, max y sum) pero al analizar el código se puede interpretar que el funcionamiento de estas varía en función al tipo que estés usando, devolviendo el valor de cada heurística al valor correspondiente de la distancia a la esquina (la distancia máxima, la mínima o la suma de todas). Para el caso de *Acomplex* sí que tiene en cuenta un mayor número de parámetros como por ejemplo las esquinas ya ocupadas, la penalización en caso de una distancia grande a las esquinas, o por el contrario la distancia a la esquina más cercana.